

CENG 467 Natural Language Understanding and Generation

Take-Home Midterm Report

Samet Buldanlıoğlu
Student ID: 300201085

May 6, 2026

Repository / Colab Link: <https://github.com/sametceng/NLU-NLG-Experiments>

Abstract

This report presents five experiments covering major natural language understanding and generation tasks: text classification, named entity recognition, text summarization, machine translation, and language modeling. The experiments compare classical, neural, and transformer-based approaches using benchmark datasets and standard evaluation metrics. All experiments were conducted with a fixed random seed of 42 where applicable, and subsets were used in some tasks for computational feasibility. The goal of the report is not only to compare performance, but also to analyze model behavior, error patterns, and methodological trade-offs.

Contents

1	Question 1 – Representation Learning in Text Classification	4
1.1	Dataset Description	4
1.2	Preprocessing and Tokenization Strategies	4
1.3	Models	4
1.4	Final Results	5
1.5	Misclassification Analysis	5
1.6	Discussion	6
2	Question 2 – Named Entity Recognition	6
2.1	Dataset Description and Annotation Structure	6
2.2	Token-Label Alignment	7
2.3	Models	7
2.4	Results	8
2.5	CRF Detailed Results	8
2.6	Error Analysis	8
2.7	Discussion	9

3	Question 3 – Text Summarization	9
3.1	Dataset Description	9
3.2	Methods	10
3.3	TextRank Extractive Summarization	10
3.4	BART Abstractive Summarization	10
3.5	Evaluation Metrics	11
3.6	Results	11
3.7	Qualitative Analysis	11
3.7.1	Example 1	11
3.7.2	Example 2	12
3.7.3	Example 3	12
3.8	Discussion	12
4	Question 4 – Machine Translation	13
4.1	Dataset Description	13
4.2	Preprocessing	13
4.3	Models	13
4.4	Seq2Seq with Attention	13
4.5	Pretrained Transformer	14
4.6	Evaluation Metrics	14
4.7	Results	14
4.8	Qualitative Analysis	15
4.8.1	Example 1	15
4.8.2	Example 2	15
4.8.3	Example 3	15
4.9	Discussion	16
5	Question 5 – Language Modeling	16
5.1	Dataset Description	16
5.2	Preprocessing	17
5.3	Models	17
5.4	Trigram N-gram Model	17
5.5	LSTM Language Model	18
5.6	Results	18
5.7	LSTM Training Behavior	19
5.8	Generated Text Samples	19

5.8.1	Trigram Samples	19
5.8.2	LSTM Samples	19
5.9	Discussion	20
6	Conclusion	20

1 Question 1 – Representation Learning in Text Classification

1.1 Dataset Description

For this task, the SST-2 dataset was used as the benchmark sentiment classification dataset [1]. SST-2 is a binary sentiment classification dataset where each input is a sentence and each label indicates whether the sentiment is negative or positive. The labels are defined as follows:

- 0: negative
- 1: positive

To make the experiments computationally efficient and reproducible, a subset of the dataset was used. The training subset contained 3000 examples, and the validation subset contained 500 examples. A fixed random seed of 42 was used for shuffling and subset selection. All models were trained and evaluated on the same train-validation split.

1.2 Preprocessing and Tokenization Strategies

Several TF-IDF preprocessing strategies were tested to examine the effects of normalization, stopwords removal, and n-gram selection.

Table 1: TF-IDF preprocessing strategy comparison.

Strategy	Description	Accuracy	Macro-F1
Strategy 1	Unigram + bigram, no stopwords removal	0.760	0.756
Strategy 2	Unigram only, English stopwords removal	0.730	0.717
Strategy 3	Unigram + bigram + trigram, no stopwords removal	0.750	0.747
Strategy 4	Unigram + bigram, stopwords removal including no/not	0.724	0.708

Strategy 1 achieved the best TF-IDF performance. This suggests that preserving stopwords and using both unigram and bigram features is useful for sentiment classification. In particular, short expressions such as *not good*, *no fun*, and *very bad* can carry important sentiment information. Strategy 4 produced the lowest Macro-F1 score, especially reducing negative recall, which shows that removing negation words such as *no* and *not* harms sentiment classification performance.

1.3 Models

Three representation learning approaches were implemented and compared:

1. TF-IDF + Logistic Regression as a sparse lexical baseline.
2. BiLSTM as a dense sequential neural model.
3. DistilBERT as a contextual transformer-based model [5, 6].

The TF-IDF model used Strategy 1 as the final sparse baseline because it achieved the best validation performance among the TF-IDF variants. The BiLSTM model used word embeddings, a bidirectional LSTM layer, and a fully connected classification layer. DistilBERT was fine-tuned for binary sequence classification using the same train and validation subsets.

1.4 Final Results

Table 2: Question 1 final model comparison.

Model	Representation Type	Accuracy	Macro-F1
TF-IDF + Logistic Regression	Sparse lexical representation	0.760	0.756
BiLSTM	Dense sequential representation	0.640	0.610
DistilBERT	Contextual transformer representation	0.872	0.872

DistilBERT achieved the best overall performance, with 0.872 Accuracy and 0.872 Macro-F1. This result shows the advantage of contextual transformer-based representations. Unlike TF-IDF, DistilBERT can represent words according to their surrounding context. Unlike the BiLSTM model, it also benefits from large-scale pretraining.

The TF-IDF model performed reasonably well and remained a strong interpretable baseline. The BiLSTM model underperformed compared with TF-IDF and DistilBERT, likely because it was trained from scratch on a limited subset without pretrained embeddings.

1.5 Misclassification Analysis

Five misclassified examples from the best-performing DistilBERT model were analyzed.

Table 3: Misclassified examples from DistilBERT.

No.	Sentence	True	Predicted	Error Pattern
1	“you’ll gasp appalled and laugh outraged...”	positive	negative	Mixed sentiment and negative lexical cues
2	“sam mendes has become valedictorian...”	negative	positive	Implicit criticism or sarcasm
3	“not far beneath the surface...”	positive	negative	Negative-sounding vocabulary
4	“no screen fantasy-adventure...”	positive	negative	Negation-like structure misunderstood
5	“i’ll bet the video game is a lot more fun than the film.”	negative	positive	Comparative expression with positive word

These examples show that even DistilBERT struggles with subtle sentiment expressions. Several errors contain both positive and negative lexical cues. For example, words such as *appalled*, *outraged*, *nasty*, and *disturbing* may bias the model toward the negative class, even when the

gold label is positive. Another common error pattern is implicit criticism or sarcasm, where the sentence does not directly use strongly negative words but still conveys negative sentiment.

The fifth example is especially interesting because the word *fun* is positive, but the sentence actually criticizes the film by saying that the video game is more fun than the film. This shows that comparative structures can still be difficult for contextual models.

1.6 Discussion

The results demonstrate clear differences between sparse, dense, and contextual representations. TF-IDF is simple, efficient, and interpretable because each feature corresponds to a word or n-gram. However, it has limited ability to capture deeper context. BiLSTM can model word order and sequential dependencies, but it requires more data and careful tuning when trained from scratch. DistilBERT achieved the strongest performance because it uses pretrained contextual embeddings.

Overall, contextual transformer representations provided the best generalization performance, while TF-IDF remained a strong and interpretable baseline. The BiLSTM model showed the limitations of training a neural sequence model from scratch on a small subset.

2 Question 2 – Named Entity Recognition

2.1 Dataset Description and Annotation Structure

For this task, the CoNLL-2003 dataset was used for named entity recognition [2]. The dataset consists of tokenized sentences, where each token is annotated with a named entity label. Each example contains tokens, part-of-speech tags, chunk tags, and named entity tags. In this experiment, only the `tokens` and `ner_tags` fields were used.

Table 4: CoNLL-2003 dataset split sizes.

Split	Number of Sentences
Train	14,041
Validation	3,250
Test	3,453

To keep the experiments computationally efficient, a subset was used.

Table 5: Used subset sizes for NER experiments.

Split	Used Size
Train	3,000
Validation	800

The named entity labels follow the BIO tagging scheme:

['O', 'B-PER', 'I-PER', 'B-ORG', 'I-ORG', 'B-LOC', 'I-LOC', 'B-MISC', 'I-MISC']

Table 6: BIO label meanings.

Label	Meaning
O	Outside any named entity
B-PER	Beginning of a person entity
I-PER	Inside a person entity
B-ORG	Beginning of an organization entity
I-ORG	Inside an organization entity
B-LOC	Beginning of a location entity
I-LOC	Inside a location entity
B-MISC	Beginning of a miscellaneous entity
I-MISC	Inside a miscellaneous entity

For example:

Table 7: Example token-label pairs.

Token	Label
EU	B-ORG
rejects	O
German	B-MISC
call	O
to	O
boycott	O
British	B-MISC
lamb	O
.	O

2.2 Token-Label Alignment

For the CRF model, token-label alignment is direct because each original token corresponds to exactly one BIO label.

For the BERT-based model, token-label alignment is more complex because BERT uses subword tokenization. A word may be split into multiple subword tokens. In this case, the gold label was assigned only to the first subword token, and the remaining subwords were assigned -100, which is ignored during loss computation. This ensures that evaluation remains aligned with the original word-level labels.

2.3 Models

Two models were implemented and compared:

1. CRF as a classical sequence labeling model.
2. BERT token classification as a transformer-based contextual model [5].

The CRF model used handcrafted features such as lowercased word form, suffixes, capitalization, digit information, previous word features, and next word features. The BERT model used `bert-base-cased`, which is appropriate for NER because capitalization is an important signal for named entities.

2.4 Results

Table 8: Question 2 NER results.

Model	Precision	Recall	F1-score
CRF	0.837	0.783	0.809
BERT	0.870	0.886	0.878

BERT achieved the best overall performance with an F1-score of 0.878, compared with 0.809 for CRF. BERT also achieved higher recall, meaning it detected more entity spans than CRF.

The CRF model still performed reasonably well, especially considering that it relies on handcrafted features rather than pretrained contextual embeddings. However, its lower recall suggests that it missed more entities.

2.5 CRF Detailed Results

Table 9: CRF entity-level results.

Entity Type	Precision	Recall	F1-score
LOC	0.87	0.80	0.83
MISC	0.83	0.75	0.79
ORG	0.75	0.68	0.71
PER	0.86	0.86	0.86

The CRF model performed best on `PER` entities and weakest on `ORG` entities. Organization names are often more difficult because they can have diverse surface forms and may overlap with location or miscellaneous entity patterns.

2.6 Error Analysis

Five misclassified examples from the BERT model were analyzed.

Table 10: BERT NER error examples.

No.	Sentence	Error Type
1	“Pro-Moscow leaders in Chechnya...”	Entity type confusion
2	“Singapore hanged a Thai farmer at Changi Prison...”	Boundary error
3	“Bristol : Gloucestershire...”	Entity type confusion
4	“UNITED NATIONS 1996-08-29”	Multi-token entity confusion
5	“The Dow Jones industrial average...”	Entity type confusion

In the first example, *Pro-Moscow* was missed completely, and *Chechnya* was predicted as **MISC** instead of **LOC**. This shows both false negative and entity type confusion errors.

In the second example, the model failed to label *Prison* as part of *Changi Prison*, which is a boundary error. In the fourth example, *UNITED NATIONS* was inconsistently labeled across tokens, showing a multi-token consistency problem.

2.7 Discussion

The results show that contextual embeddings are highly beneficial for NER. BERT represents each token based on the full sentence context, which helps disambiguate entity types. For example, a word such as *Bristol* may refer to a location or organization depending on context.

However, BERT still made errors involving boundary detection, entity type confusion, and multi-token consistency. This shows that NER is not only about detecting entity tokens but also about correctly identifying entity spans and maintaining consistent BIO labels.

Overall, BERT provided stronger generalization, while CRF remained a competitive and interpretable classical baseline.

3 Question 3 – Text Summarization

3.1 Dataset Description

For this task, the CNN/DailyMail dataset was used to compare extractive and abstractive summarization methods [8]. Each example contains a long news article and a human-written highlight summary.

Each example includes:

- **article:** full news article
- **highlights:** reference summary
- **id:** document identifier

Table 11: CNN/DailyMail dataset split sizes.

Split	Number of Examples
Train	287,113
Validation	13,368
Test	11,490

A subset of 20 validation examples was used for computational feasibility. The same subset was used for both TextRank and BART.

3.2 Methods

Two summarization methods were implemented:

1. TextRank as an extractive method.
2. BART-large-CNN as an abstractive method [7].

3.3 TextRank Extractive Summarization

TextRank is a graph-based ranking method inspired by PageRank [9]. Each sentence is treated as a node, and sentence similarity is computed using TF-IDF and cosine similarity. PageRank is then applied to rank sentences by importance. The top three ranked sentences are selected and ordered according to their original article positions.

The TextRank pipeline was:

1. Split article into sentences.
2. Represent sentences using TF-IDF.
3. Compute cosine similarity between sentences.
4. Build a sentence similarity graph.
5. Apply PageRank.
6. Select top 3 sentences.
7. Preserve original order.

Since TextRank is extractive, it copies sentences directly from the source article. This improves factual faithfulness but may reduce fluency and compactness.

3.4 BART Abstractive Summarization

For abstractive summarization, `facebook/bart-large-cnn` was used. BART is a pretrained sequence-to-sequence transformer model. Unlike TextRank, BART can generate new sentences and paraphrase the source article.

Table 12: BART summarization setup.

Parameter	Value
Model	facebook/bart-large-cnn
Input truncation	1024 tokens
Max summary length	130 tokens
Min summary length	30 tokens
Beam search	4 beams
Sampling	Disabled

3.5 Evaluation Metrics

The summaries were evaluated using ROUGE-1, ROUGE-2, ROUGE-L, BLEU, METEOR, and BERTScore.

Table 13: Summarization evaluation metrics.

Metric	Purpose
ROUGE-1	Unigram overlap
ROUGE-2	Bigram overlap
ROUGE-L	Longest common subsequence
BLEU	Translation-style n-gram precision
METEOR	Alignment, stemming, and synonym matching
BERTScore	Contextual semantic similarity

3.6 Results

Table 14: Question 3 summarization results.

Method	ROUGE-1	ROUGE-2	ROUGE-L	BLEU	METEOR	BERTScore-F1
TextRank	0.351	0.139	0.233	0.096	0.323	0.866
BART	0.397	0.172	0.281	0.163	0.337	0.875

BART outperformed TextRank across all automatic metrics. It achieved higher ROUGE, BLEU, METEOR, and BERTScore-F1 values. This suggests that BART summaries were closer to the reference summaries both lexically and semantically.

3.7 Qualitative Analysis

3.7.1 Example 1

Reference Summary

Jarryd Hayne quit the NRL in October to try and get into American Football. This week, he signed a three-year contract with the San Francisco 49ers.

The chairman of the US Association of Rugby League welcomed his arrival.

TextRank Summary

The Australia international full-back or centre quit the National Rugby League in October to try his luck in American football and was this week given a three-year contract with the San Francisco 49ers. Hayne, who played rugby league for Australia, has signed a three year contract with the 49ers. Hayne could play at full back or centre in rugby league and is expected to be a running back for the 49ers.

BART Summary

Jarryd Hayne quit the NRL in October to try his luck in American football. The 27-year-old has signed a three-year contract with the San Francisco 49ers. Hayne could play at full back or centre in rugby league. He is expected to be a running back for the 49ers.

Both models captured the main event. However, TextRank produced a more repetitive and extractive summary, while BART generated a more fluent and compact summary closer to the reference style.

3.7.2 Example 2

TextRank selected relevant sentences but included long source fragments. BART produced a more coherent narrative covering Faith March's anorexia, hospital treatment, and recovery through her patisserie business. BART was more fluent, while TextRank was more directly faithful to the source.

3.7.3 Example 3

TextRank selected a sentence beginning with "ET Tuesday," which is not fully self-contained. This shows a common weakness of extractive summarization: selected sentences may depend on previous context. BART produced a cleaner and more readable summary about *Community* returning for its sixth season on Yahoo.

3.8 Discussion

TextRank is computationally cheaper, more transparent, and generally faithful because it directly extracts source sentences. However, it can produce summaries that are less fluent, overly long, or contextually incomplete.

BART is more computationally expensive but produces more fluent and compact summaries. It can paraphrase and combine information from different parts of the article. However, as an abstractive model, it may carry a higher risk of hallucination.

In this experiment, BART achieved better automatic scores and more readable summaries. TextRank remained useful as a faithful and interpretable extractive baseline.

4 Question 4 – Machine Translation

4.1 Dataset Description

For this task, the Multi30k dataset was used for English-to-German machine translation [3]. Multi30k consists of short image-caption style sentence pairs.

The translation direction was English to German. Each example contains an English source sentence and a German reference translation.

A subset of the dataset was used for computational feasibility. The same validation examples were used for both the Seq2Seq with Attention model and the pretrained Transformer model.

4.2 Preprocessing

For the Seq2Seq model, a simple preprocessing pipeline was used:

1. Lowercase text.
2. Separate punctuation.
3. Normalize whitespace.
4. Tokenize by whitespace.
5. Build source and target vocabularies.
6. Add <pad>, <sos>, <eos>, and <unk>.
7. Truncate or pad sequences to 40 tokens.

For the pretrained Transformer model, the model’s own tokenizer was used.

4.3 Models

Two translation models were compared:

1. Seq2Seq with Attention
2. Pretrained Transformer-based MarianMT model

4.4 Seq2Seq with Attention

The Seq2Seq model used a bidirectional GRU encoder and a GRU decoder with additive attention. The attention mechanism allows the decoder to focus on different encoder states while generating each target token.

Table 15: Seq2Seq with attention setup.

Component	Setting
Encoder	Bidirectional GRU
Decoder	GRU
Attention	Additive attention
Embedding dimension	128
Hidden dimension	256
Dropout	0.3
Epochs	5
Teacher forcing ratio	0.5
Optimizer	Adam

4.5 Pretrained Transformer

The pretrained model used was `Helsinki-NLP/opus-mt-en-de`, which is a MarianMT model pretrained on large-scale English-German parallel corpora [11]. Its subword tokenizer reduces unknown word problems and improves handling of rare words.

4.6 Evaluation Metrics

Table 16: Machine translation evaluation metrics.

Metric	Purpose
BLEU	N-gram precision with brevity penalty
METEOR	Alignment and synonym-aware similarity
ChrF	Character n-gram F-score
BERTScore	Contextual semantic similarity

4.7 Results

Table 17: Question 4 machine translation results.

Model	BLEU	METEOR	ChrF	BERTScore-F1
Seq2Seq + Attention	1.645	0.396	29.297	0.685
Pretrained Transformer	36.511	0.647	63.652	0.895

The pretrained Transformer clearly outperformed the Seq2Seq with Attention model across all metrics. This large gap is expected because the Seq2Seq model was trained from scratch on a limited subset, while the Transformer was pretrained on large-scale parallel data.

4.8 Qualitative Analysis

4.8.1 Example 1

Source

2 girls playing volleyball, one striking the ball.

Reference

Zwei Mdchen spielen Volleyball, wobei das eine den Ball schltgt.

Seq2Seq Output

<unk> mdchen spielen <unk> <unk> den ball .

Transformer Output

2 Mdchen spielen Volleyball, eine schltgt den Ball.

The Seq2Seq model captured some words but produced several <unk> tokens. The Transformer produced a fluent and semantically accurate translation.

4.8.2 Example 2

Source

A man and woman are walking down a waterside path toward a suspension bridge.

Reference

Ein Mann und eine Frau gehen einen Uferweg entlang auf eine Hngebrcke zu.

Seq2Seq Output

ein mann und eine frau gehen gehen einen <unk> entlang , die eine <unk> entlang .

Transformer Output

Ein Mann und eine Frau gehen einen Wasserweg hinunter zu einer Hngebrcke.

The Seq2Seq output contains repetition and unknown tokens. The Transformer output is more fluent and preserves the general meaning.

4.8.3 Example 3

Source

Two man are paddling a kayak along a river with green trees on either side.

Reference

Zwei Mnner paddeln in einem Kajak einen Fluss mit grnen Bumen auf beiden Seiten entlang.

Seq2Seq Output

zwei mann mit einem einem mit einem fahrrad mit einem <unk> mit einem <unk> .

Transformer Output

Zwei Mann paddeln ein Kajak entlang eines Flusses mit grnen Bumen auf beiden Seiten.

The Seq2Seq model produced repetition and incorrect lexical choices such as *fahrrad*, which does not appear in the source. The Transformer preserved most of the meaning, although *Mann* should ideally be *Männer*.

4.9 Discussion

The Seq2Seq with Attention model is useful for understanding sequence-to-sequence modeling and attention mechanisms. However, it struggled with unknown words, repetition, rare words, and long-range dependencies.

The pretrained Transformer performed much better because it benefits from large-scale pre-training, subword tokenization, and self-attention. It produced more fluent, complete, and semantically faithful translations.

BLEU reflected the large difference in n-gram overlap. ChrF was useful because German is morphologically rich and character-level overlap captures partial word-form similarity. METEOR rewarded better alignments, while BERTScore showed stronger semantic similarity for the Transformer outputs.

5 Question 5 – Language Modeling

5.1 Dataset Description

For this task, the WikiText-2 dataset was used for language modeling [4]. WikiText-2 is a word-level language modeling dataset constructed from Wikipedia articles. The objective is to model the probability of token sequences and predict the next token given previous context.

Table 18: WikiText-2 dataset split sizes.

Split	Number of Lines
Train	36,718
Validation	3,760
Test	4,358

After preprocessing, the token counts were:

Table 19: WikiText-2 token counts after preprocessing.

Split	Number of Tokens
Train	447,047
Validation	47,550
Test	59,678

A vocabulary of 10,000 tokens was built from the training set. Rare or unseen words were mapped to `<unk>`.

5.2 Preprocessing

The preprocessing pipeline was:

1. Convert text to lowercase.
2. Remove empty lines.
3. Remove WikiText section markers.
4. Tokenize using word-level tokenization.
5. Build vocabulary from the training set.
6. Replace out-of-vocabulary words with `<unk>`.

For LSTM training, sequences were divided into segments of 35 tokens, where the input sequence is used to predict the next-token-shifted target sequence.

5.3 Models

Two language models were implemented:

1. Trigram N-gram Language Model
2. LSTM Language Model

5.4 Trigram N-gram Model

The trigram model estimates:

$$P(w_i \mid w_{i-2}, w_{i-1}) \tag{1}$$

Add-k smoothing was used with $\alpha = 0.1$. The smoothed probability is:

$$P(w \mid context) = \frac{\text{count}(context, w) + \alpha}{\text{count}(context) + \alpha|V|} \tag{2}$$

This model is simple and interpretable, but it only uses the previous two words as context.

5.5 LSTM Language Model

The LSTM model used distributed word embeddings and recurrent hidden states to model longer context.

Table 20: LSTM language model setup.

Component	Setting
Embedding dimension	128
Hidden dimension	256
Number of LSTM layers	2
Dropout	0.3
Sequence length	35
Batch size	64
Epochs	5
Optimizer	Adam
Loss function	CrossEntropyLoss

Perplexity was computed as:

$$Perplexity = \exp(loss) \quad (3)$$

Lower perplexity indicates better language modeling performance.

5.6 Results

Table 21: Question 5 language modeling results.

Model	Validation Perplexity	Test Perplexity
Trigram N-gram	2205.38	2305.19
LSTM	147.38	148.30

The LSTM language model substantially outperformed the trigram N-gram model. The trigram model produced very high perplexity because it relies only on the previous two tokens and suffers from data sparsity. The LSTM achieved much lower perplexity because it can learn longer sequential dependencies.

5.7 LSTM Training Behavior

Table 22: LSTM training behavior across epochs.

Epoch	Train Perplexity	Validation Perplexity
1	211.13	147.38
2	80.70	162.28
3	55.60	188.77
4	44.78	218.36
5	38.67	247.27

Although training perplexity continuously decreased, validation perplexity increased after the first epoch. Therefore, the checkpoint from epoch 1 was selected as the best model. This indicates overfitting: the model learned the training data more closely but generalized worse to unseen data.

5.8 Generated Text Samples

5.8.1 Trigram Samples

Prompt: the

```
the eyes everett benjamin ballcourt likely analysis does attitudes culmination tony
january vaguely coinage beyonce 're aspects drawing guards 59 portion poetry mix
watch median subcontinent serialised elected Nile Cheltenham republic
```

Prompt: in the

```
in the 1929 egyptians advantage governess midway vertically thaksin dock shoop hudson
martial grassland stated understated funds miracle camera users 16th agency
transcend devastating potato humidity scandal ilp itself transportation castle
princeton
```

Prompt: he

```
he ha height edinburgh accordingly flat maximus sb flattened involved acorn slipped
spread have omniscient restaurant cruelty romanesque appointed statues hit
prolonged solar requested books bringing chicks forget limantour cooled uniform
```

The trigram model generated mostly incoherent word sequences. Some local combinations may appear plausible, but the output lacks grammatical structure and semantic coherence.

5.8.2 LSTM Samples

Prompt: the

```
the other , the u.s. god who had begun a <unk> group . that also did not <unk> to <unk>
> their <unk> from the <unk> . <unk> enjoyed a series of
```

Prompt: in the

in the establishment of <unk> . he is three support for the state of the church in <unk> and <unk> . the army was <unk> to <unk> . this team were ruled

Prompt: he

he recovered to experience in the city and 1855 of a link of the <unk> flight of orbit and the french <unk> . on december 28 , 2007 , the final

The LSTM generated more fluent and sentence-like text than the trigram model. It produced more recognizable phrase structures and punctuation. However, it still contained <unk> tokens and semantic inconsistencies, showing limitations in vocabulary coverage and long-range coherence.

5.9 Discussion

The trigram model is simple and interpretable, but it suffers from data sparsity and limited context. It cannot model long-range dependencies and therefore produces incoherent text with very high perplexity.

The LSTM model performs much better because it learns dense word representations and maintains a hidden state over time. This allows it to capture longer context and generate more fluent text. However, it also showed overfitting, and its generated samples still contained unknown tokens and semantic inconsistencies.

Overall, the LSTM provided much stronger probabilistic sequence modeling than the trigram baseline, while the trigram model served as a simple interpretable baseline.

6 Conclusion

Across the five tasks, transformer-based or pretrained models generally achieved the strongest performance. DistilBERT outperformed sparse and recurrent models in sentiment classification, BERT improved NER performance over CRF, BART produced stronger summaries than TextRank, and the pretrained MarianMT Transformer substantially outperformed a Seq2Seq model trained from scratch. In language modeling, the LSTM model substantially outperformed the trigram baseline, although it still showed overfitting and vocabulary limitations.

These results show that pretrained contextual models provide strong generalization across NLU and NLG tasks. However, classical models such as TF-IDF, CRF, TextRank, and N-gram models remain useful as interpretable baselines and help reveal the strengths and limitations of more complex neural approaches.

References

- [1] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Y. Ng, and Christopher Potts. Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pages 1631–1642, 2013.
- [2] Erik F. Tjong Kim Sang and Fien De Meulder. Introduction to the CoNLL-2003 Shared Task: Language-Independent Named Entity Recognition. In *Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003*, pages 142–147, 2003.

- [3] Desmond Elliott, Stella Frank, Khalil Sima'an, and Lucia Specia. Multi30K: Multilingual English-German Image Descriptions. In *Proceedings of the 5th Workshop on Vision and Language*, pages 70–74, 2016.
- [4] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer Sentinel Mixture Models. *arXiv preprint arXiv:1609.07843*, 2016.
- [5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.
- [6] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter. *arXiv preprint arXiv:1910.01108*, 2019.
- [7] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7871–7880, 2020.
- [8] Abigail See, Peter J. Liu, and Christopher D. Manning. Get To The Point: Summarization with Pointer-Generator Networks. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, pages 1073–1083, 2017.
- [9] Rada Mihalcea and Paul Tarau. TextRank: Bringing Order into Text. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, pages 404–411, 2004.
- [10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. In *Advances in Neural Information Processing Systems*, 2017.
- [11] Marcin Junczys-Dowmunt, Roman Grundkiewicz, Tomasz Dwojak, Hieu Hoang, Kenneth Heafield, Tom Neckermann, Frank Seide, Ulrich Germann, Alham Fikri Aji, Nikolay Bogoychev, André F. T. Martins, and Alexandra Birch. Marian: Fast Neural Machine Translation in C++. In *Proceedings of ACL 2018, System Demonstrations*, pages 116–121, 2018.
- [12] Chin-Yew Lin. ROUGE: A Package for Automatic Evaluation of Summaries. In *Text Summarization Branches Out*, pages 74–81, 2004.
- [13] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: a Method for Automatic Evaluation of Machine Translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, 2002.
- [14] Satanjeev Banerjee and Alon Lavie. METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments. In *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, pages 65–72, 2005.
- [15] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. BERTScore: Evaluating Text Generation with BERT. In *International Conference on Learning Representations*, 2020.
- [16] Maja Popović. chrF: character n-gram F-score for automatic MT evaluation. In *Proceedings of the Tenth Workshop on Statistical Machine Translation*, pages 392–395, 2015.